AppViajes

Especificación de requisitos de software · Entrega 1

Proyecto integrador ExpoUninpahu 2026 · Ingeniería de Software

Integrantes: Johnny Esteban Buitrago Ortiz, Cristian Camilo Pachon Roman y Yulieth Alejandra Leon Pineda.

Docente: Elfar Didier Morantes Sánchez
Fecha de entrega: 05 de noviembre de 2026

1 Introducción

1.1 Propósito

Definir las funciones, el alcance y los requisitos de AppViajes como proyecto de la clase de Django. Este documento orienta el desarrollo y la comprobación de la aplicación.

1.2 Problema

Quién tiene el problema: grupos de amigos o familias que organizan un viaje y deben decidir entre varios destinos o planes.
Cómo se ve hoy: las alternativas, los presupuestos y las preferencias se dispersan en chats de WhatsApp, mensajes y hojas de cálculo; las votaciones se hacen a mano y no queda un registro único que permita comparar.
Por qué vale la pena resolverlo: centralizar destinos, presupuesto en COP y votación en un solo lugar reduce el tiempo de decisión y evita elegir sin información.

Solución propuesta: La aplicación permitirá consultar esa información y los resultados de la votación en un mismo lugar, facilitando la elección del grupo.

2 Justificación

El proyecto permite aplicar Django en una solución que integra formularios, base de datos, votaciones y autenticación. La conexión con un servicio Node.js y una API de IA ampliará la aplicación con sugerencias de itinerarios.

3 Objetivos

Objetivo general: desarrollar una plataforma web para registrar destinos con presupuesto estimado en COP, definir opciones de plan, votar, consultar resultados y obtener sugerencias de itinerarios mediante IA.

Objetivos específicos:
1. Gestionar destinos, opciones de plan, votos y resultados.
2. Incorporar inicio de sesión y permisos por rol.
3. Conectar Django con el servicio Node.js y la API de IA.
4. Ejecutar el proyecto con Docker, desplegarlo y comprobar sus funciones mediante pruebas.

4 Alcance

Incluye: CRUD de destinos; definición y consulta de opciones; votación y resultados; registro de cuentas, autenticación y roles; sugerencias de itinerarios; PostgreSQL; Docker y despliegue en Render.

No incluye: compra de tiquetes, reservas, pagos ni una aplicación móvil nativa.

Estado actual: existe una aplicación Django funcional en entorno local con CRUD de destinos, definición de opciones, votación mediante contador por opción y resultados, probada con tests. Todavía no hay registro de usuarios, inicio de sesión, roles, dueño de cada destino ni voto individual por usuario; estos se implementan en la Entrega 2. Faltan también Docker, PostgreSQL, microservicio Node.js, integración de IA y despliegue.

5 Tecnologías

Tecnología	Uso en el proyecto
Django	Aplicación web, autenticación, permisos, destinos, opciones, votación y resultados.
Node.js y Express	Microservicio ia-service con API REST y conexión con la IA.
Swagger y OpenAPI	Documentación de la API del microservicio.
API de IA generativa	Sugerencias de itinerarios. Proveedor y modelo pendientes de selección.
PostgreSQL	Almacenamiento de los datos de la aplicación.
Docker y Docker Compose	Ejecución local de los servicios web e ia-service.
Render	Despliegue de Django, Node.js y PostgreSQL administrado.
GitHub y Git Flow	Repositorio y organización de las ramas.

Django atenderá al navegador y guardará los datos en PostgreSQL. Para generar un itinerario, su cliente HTTP consultará a Node.js; este consumirá la API de IA y devolverá la respuesta a Django.

6 Definiciones

SRS: especificación de requisitos de software. RF: requisito funcional. RNF: requisito no funcional. CU: caso de uso. CRUD: crear, consultar, actualizar y eliminar. COP: peso colombiano. Destino: registro con nombre, descripción y presupuesto estimado. Opción de plan: alternativa asociada a un destino. Itinerario: propuesta de actividades para el viaje.

7 Actores

Actor	Descripción
Visitante	Consulta destinos, opciones y resultados; puede registrarse e iniciar sesión.
Viajero	Crea destinos y opciones, vota y solicita itinerarios. Modifica sus propios destinos.
Administrador	Gestiona todos los registros y usuarios; utiliza las funciones del viajero.
API de IA externa	Servicio que genera las sugerencias de itinerarios.

El viajero solo podrá modificar sus propios destinos y definir sus opciones. El administrador podrá gestionar todos los registros y usuarios, además de utilizar las funciones del viajero.

8 Restricciones y supuestos

La solución debe utilizar las tecnologías de la sección 5. Los presupuestos se expresarán en COP. Las claves de los servicios se configurarán en el servidor. El usuario necesitará conexión a Internet y un navegador; se supone que el equipo tendrá acceso al proveedor de IA y a la plataforma de despliegue.

Plan de entregas: E1, 05/11/2026: documentación (SRS, UML, backlog, trazabilidad) y funciones base existentes en entorno local. E2, 12/11/2026: registro, login, roles, reglas de votación (un voto por usuario y destino), Django Admin, PostgreSQL, Dockerfile de Django, aplicación desplegada públicamente (despliegue inicial en Render) y primera versión del diagrama de secuencia. E3, 19/11/2026: microservicio Node.js desplegado, Swagger/OpenAPI público, integración con IA, Docker Compose y UML actualizado. ExpoUninpahu, 26/11/2026: producto final y sustentación.

Reglas: un voto por usuario y destino, con posibilidad de cambiarlo; al eliminar un destino se eliminan sus opciones y votos. Los campos de los formularios serán obligatorios y el presupuesto deberá ser mayor que cero; esta validación se aplica tanto en el modelo como en el formulario de la aplicación. Para generar un itinerario se indicarán destino, presupuesto y número de días.

9 Requisitos funcionales

Prioridad Alta: función necesaria para el proyecto. E1 identifica la documentación y las funciones base existentes en entorno local. E2 incorpora registro, login, roles, reglas de votación, Django Admin, PostgreSQL, Dockerfile de Django, el despliegue público inicial en Render y la primera versión del diagrama de secuencia. E3 incorpora el microservicio Node.js desplegado, Swagger/OpenAPI público, la integración con IA, Docker Compose y UML actualizado.

ID	Descripción	Prioridad	Entrega
RF-01	Crear destinos con nombre, descripción y presupuesto estimado en COP.	Alta	E1
RF-02	Consultar los destinos y sus datos.	Alta	E1
RF-03	Actualizar destinos propios; el administrador puede actualizar cualquiera.	Alta	E2
RF-04	Eliminar un destino junto con sus opciones y votos, según los permisos del rol.	Alta	E2
RF-05	Definir opciones para destinos propios; el administrador puede hacerlo en cualquiera.	Alta	E2
RF-06	Consultar las opciones del destino seleccionado.	Alta	E1
RF-07	Emitir un voto por usuario y destino, con posibilidad de cambiarlo.	Alta	E2
RF-08	Mostrar la cantidad de votos por opción de un destino.	Alta	E1
RF-09	Iniciar sesión con una cuenta registrada y credenciales válidas.	Alta	E2
RF-10	Cerrar la sesión del usuario.	Alta	E2
RF-11	Aplicar los permisos de consulta, gestión y participación de la sección 7.	Alta	E2
RF-12	Mostrar un itinerario según destino, presupuesto y días, mediante Node.js e IA, sin guardarlo.	Alta	E3
RF-13	Validar campos obligatorios y presupuesto mayor que cero.	Alta	E2 / E3
RF-14	Registrar una cuenta con el rol de viajero.	Alta	E2
RF-15	Gestionar usuarios, destinos, opciones y votos desde el panel administrativo de Django Admin.	Alta	E2

En la Entrega 1 existen en entorno local las versiones base de RF-01, RF-02, RF-06 y RF-08; las funciones que dependen de usuario, dueño o rol se completan en la Entrega 2.

10 Requisitos no funcionales

ID	Categoría	Requisito y comprobación
RNF-01	Seguridad	Comprobar permisos en el servidor y proteger contraseñas y claves. Verificar accesos permitidos y denegados.
RNF-02	Rendimiento	La generación de itinerarios no deberá bloquear las consultas de destinos y resultados. Verificar con solicitudes simultáneas.
RNF-03	Disponibilidad	Ante fallos de la IA, informar el error y mantener las funciones independientes. Comprobar con un fallo simulado.
RNF-04	Portabilidad	Construir e iniciar el entorno con Docker Compose. Comprobar desde una copia limpia del repositorio.

ID	Categoría	Requisito y comprobación
RNF-05	Mantenibilidad	Separar Django y Node.js, documentar la ejecución y conservar pruebas. Ejecutar los tests antes de entregar.
RNF-06	Usabilidad	Presentar formularios claros, presupuestos en COP y mensajes de validación. Revisar los recorridos principales.
RNF-07	Interoperabilidad	Intercambiar JSON entre Django y Node.js y documentar la API con OpenAPI. Comprobar solicitudes y respuestas.
RNF-08	Configuración	Las claves de API, la clave secreta, las credenciales de la base de datos y las URLs entre servicios se leen de variables de entorno y no se guardan en el repositorio. Comprobar que el repositorio no contiene secretos y que existe un archivo de ejemplo de variables.

11 Casos de uso

11.1 Listado

El visitante consulta destinos, opciones y resultados. Las operaciones de escritura requieren sesión: el viajero gestiona sus propios destinos y el administrador puede gestionar todos.

ID	Caso de uso	Actor	RF asociados
CU-01	Registrar destino	Viajero o administrador	01, 11, 13
CU-02	Consultar destinos	Visitante, viajero o administrador	02, 11
CU-03	Actualizar destino	Viajero o administrador	03, 11, 13
CU-04	Eliminar destino	Viajero o administrador	04, 11
CU-05	Definir opciones de plan	Viajero o administrador	05, 11, 13
CU-06	Consultar opciones de plan	Visitante, viajero o administrador	06, 11
CU-07	Votar por una opción	Viajero o administrador	07, 11, 13
CU-08	Consultar resultados	Visitante, viajero o administrador	08, 11
CU-09	Iniciar sesión	Visitante con cuenta	09
CU-10	Cerrar sesión	Viajero o administrador	10
CU-11	Solicitar itinerario sugerido	Viajero o administrador; apoyo de API de IA externa	12, 11, 13
CU-12	Registrar cuenta	Visitante	14
CU-13	Administrar el sistema desde el panel administrativo	Administrador	15, 11

En la última columna, 01 corresponde a RF-01 y así sucesivamente.

11.2 CU 01 Registrar destino

Actor: viajero o administrador habilitado. Requisitos: RF-01, RF-11 y RF-13.

Precondición: el actor cuenta con el permiso para crear destinos.

Flujo principal:
1. El actor abre el formulario de destino.
2. Ingresa nombre, descripción y presupuesto estimado en COP.
3. Envía el formulario.
4. El sistema valida los datos, guarda el destino y confirma el registro.

Flujo alterno: si los datos son inválidos, el sistema indica qué debe corregirse y no guarda el registro. Si no existe permiso o falla el guardado, informa el rechazo o el error.

Postcondición: el destino queda registrado y disponible para consulta. Si la operación falla, no se presenta una confirmación de éxito.

11.3 CU 07 Votar por una opción

Actor: viajero o administrador con sesión activa. Requisitos: RF-07, RF-11 y RF-13.

Precondición: el usuario inició sesión y existe un destino con opciones disponibles.

Flujo principal:
1. El actor consulta las opciones de un destino.
2. Selecciona una opción y envía el voto.
3. El sistema verifica la opción y si el usuario ya tiene un voto en ese destino.
4. Registra el voto o cambia la selección anterior y confirma la operación.

Flujo alterno: si la opción es inválida o falta la sesión, se rechaza la solicitud. Si se reenvía la misma selección, se conserva el voto existente; no se añade otro voto.

Postcondición: el usuario conserva un único voto por destino. Al cambiar de opción, los resultados descuentan el voto anterior y suman el nuevo.

11.4 CU 11 Solicitar itinerario sugerido

Actor: viajero o administrador habilitado. Actor de apoyo: API de IA externa. Requisitos: RF-12, RF-11 y RF-13.

Precondición: existe el destino, el actor inició sesión y los servicios están configurados. La solicitud incluye destino, presupuesto y número de días.

Flujo principal:
1. El actor elige el destino, indica presupuesto y número de días y solicita un itinerario.
2. Django valida los datos y los envía a la API REST de Node.js.
3. Node.js consulta al proveedor de IA.
4. La API de IA devuelve la sugerencia a Node.js.
5. Node.js entrega la respuesta a Django.
6. Django muestra el itinerario como una sugerencia generada por IA.

Flujo alterno: si los datos son inválidos o falta permiso, se rechaza la solicitud. Si la IA falla o no responde, se muestra un mensaje de error y se mantienen disponibles destinos, votación y resultados.

Postcondición: el usuario visualiza la sugerencia, que no se almacena en la base de datos.

12 Historias de usuario

ID	Historia de usuario	RF asociado	CU asociado	Criterios de aceptación	Entrega
HU-01	Como viajero quiero crear destinos con nombre, descripción y presupuesto en COP para proponer alternativas de viaje.	RF-01	CU-01	1. Guarda los tres datos ingresados. 2. El destino aparece en la consulta.	E1
HU-02	Como visitante quiero consultar los destinos y sus datos para comparar las alternativas del viaje.	RF-02	CU-02	1. Muestra los destinos registrados. 2. Presenta nombre, descripción y presupuesto en COP.	E1
HU-03	Como viajero quiero actualizar mis destinos para corregir la información de mis propuestas.	RF-03	CU-03	1. Guarda los cambios en un destino propio. 2. Rechaza cambios sobre destinos ajenos. 3. El administrador puede actualizar cualquiera.	E2
HU-04	Como viajero quiero eliminar mis destinos para retirar propuestas que ya no se considerarán.	RF-04	CU-04	1. Elimina el destino, sus opciones y sus votos. 2. Rechaza la eliminación de destinos ajenos. 3. El administrador puede eliminar cualquiera.	E2
HU-05	Como viajero quiero definir opciones para mis destinos para que el grupo pueda escoger un plan.	RF-05	CU-05	1. Vincula la opción al destino elegido. 2. Solo permite crearla al propietario o al administrador.	E2
HU-06	Como visitante quiero consultar las opciones de un destino para conocer los planes disponibles.	RF-06	CU-06	1. Muestra las opciones del destino seleccionado. 2. No incluye opciones de otros destinos.	E1
HU-07	Como viajero quiero votar por una opción y poder cambiar mi selección para expresar mi preferencia actual.	RF-07	CU-07	1. Conserva un voto por usuario y destino. 2. Cambiar de opción actualiza ambos conteos. 3. Reenviar la misma selección no duplica el voto.	E2
HU-08	Como visitante quiero consultar los votos por opción para conocer las preferencias del grupo.	RF-08	CU-08	1. Muestra el conteo por opción del destino. 2. Los conteos coinciden con los votos registrados.	E1
HU-09	Como visitante quiero iniciar sesión con mi cuenta para acceder a las funciones de usuario registrado.	RF-09	CU-09	1. Las credenciales válidas abren la sesión. 2. Las credenciales inválidas no permiten el acceso.	E2
HU-10	Como viajero quiero cerrar mi sesión para finalizar mi acceso a la aplicación.	RF-10	CU-10	1. Invalida la sesión activa. 2. Las operaciones protegidas exigen iniciar sesión de nuevo.	E2

ID	Historia de usuario	RF asociado	CU asociado	Criterios de aceptación	Entrega
HU-11	Como administrador quiero que el sistema aplique permisos por rol para controlar el acceso a las operaciones.	RF-11	CU-01 CU-03 CU-04 CU-05 CU-07 CU-11 CU-13	1. El visitante solo consulta sin sesión. 2. El viajero gestiona sus propios destinos. 3. El administrador gestiona todos los registros.	E2
HU-12	Como viajero quiero solicitar un itinerario según destino, presupuesto y días para obtener una sugerencia de viaje.	RF-12	CU-11	1. La solicitud utiliza los tres datos. 2. La respuesta llega mediante Node.js e IA. 3. Muestra la sugerencia sin guardarla.	E3
HU-13	Como viajero quiero que se validen los datos que ingreso para evitar registros incompletos o presupuestos inválidos.	RF-13	CU-01 CU-03 CU-05 CU-07 CU-11	1. Rechaza campos obligatorios vacíos. 2. Rechaza presupuestos iguales o menores que cero. 3. Informa el error y no guarda datos rechazados.	E2 / E3
HU-14	Como visitante quiero registrar una cuenta para participar como viajero en la aplicación.	RF-14	CU-12	1. Crea la cuenta con el rol viajero. 2. La cuenta permite iniciar sesión.	E2
HU-15	Como administrador quiero gestionar usuarios, destinos, opciones y votos desde Django Admin para administrar la información del sistema.	RF-15	CU-13	1. Permite al administrador acceder al panel. 2. Rechaza el acceso de visitantes y viajeros. 3. Permite gestionar los cuatro tipos de registros.	E2

13 Referencias

IEEE 830 e ISO/IEC/IEEE 29148 se toman como referencias para organizar la especificación de requisitos. Los diagramas UML, el backlog y la matriz de trazabilidad conservan los identificadores RF, CU y HU de este documento.

14 Trazabilidad

Cada requisito se relaciona con caso de uso, historia de usuario, componente, evidencia en video, entrega y URL desplegada en una matriz de trazabilidad que acompaña este documento y se mantiene actualizada hasta la sustentación final.